

Aufgabe 1: AVL-Bäume**(10 Punkte)**

Fügen Sie die folgenden Zahlen nacheinander in einen *AVL-Baum* ein:

42 16 12 19 38 1

Zeichnen Sie den Baum vor und nach jeder durchgeführten Rotation. Geben Sie auch jeweils an, was für Rotationen Sie durchführen.

Aufgabe 2: Heaps**(10 Punkte)**

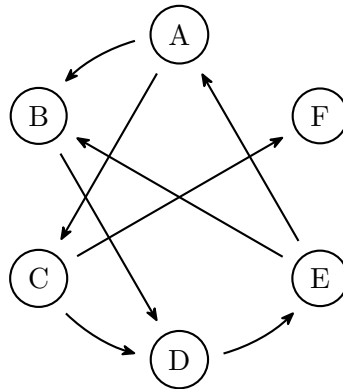
Fügen Sie die folgenden Zahlen nacheinander in einen *Min-Heap* ein:

45 18 19 7 13 2 22

Zeichnen Sie den Baum vor und nach jedem vollständigen Einfügen.

Aufgabe 3: Graphen**(10 Punkte)**

Betrachten Sie den folgenden gerichteten Graphen:



- Erstellen Sie die Adjazenzliste und die Adjazenzmatrix zu diesem Graphen.
- Welche der beiden Darstellungen ist für diesen Graphen effizienter? Begründen Sie Ihre Antwort.

Aufgabe 4: Datenstrukturen**(10 Punkte)**

Erläutern Sie die Idee und Funktionsweise des folgenden Datentyps. Diskutieren Sie kurz die Vor- und Nachteile gegenüber ähnlichen Datentypen aus der Vorlesung.

```
1  type FooType struct {
2      values    []int
3      children  []*FooType
4  }
5
6  func (f *FooType) Add(value int) {
7      if len(f.values) < 2 {
8          f.values = append(f.values, value)
9          return
10     }
11     if value < f.values[0] {
12         f.AddToChild(0, value)
13         return
14     }
15     if value < f.values[1] {
16         f.AddToChild(1, value)
17         return
18     }
19     f.AddToChild(2, value)
20 }
21
22 func (f *FooType) AddToChild(i, value int) {
23     for len(f.children) <= i {
24         f.children = append(f.children, &FooType{})
25     }
26     f.children[i].Add(value)
27 }
```

Aufgabe 5: Komplexität**(10 Punkte)**

Betrachten Sie die folgende Funktion `SameElements()`. Die Funktion erwartet zwei `int`-Listen und prüft, ob diese beiden Listen die gleiche Menge an Elementen enthalten. D.h. ob jedes Element aus der einen Liste auch in der anderen enthalten ist. Dabei spielt es keine Rolle, ob die Länge der Listen gleich ist bzw. ob die Elemente in der gleichen Anzahl vorkommen.

```
1 func SameElements(list1, list2 []int) bool {
2     for _, v1 := range list1 {
3         contained := false
4         for _, v2 := range list2 {
5             if v1 == v2 {
6                 contained = true
7             }
8         }
9         if !contained {
10            return false
11        }
12    }
13    for _, v2 := range list2 {
14        contained := false
15        for _, v1 := range list1 {
16            if v1 == v2 {
17                contained = true
18            }
19        }
20        if !contained {
21            return false
22        }
23    }
24    return true
25 }
```

- Bestimmen Sie die Komplexität dieser Funktion. Geben Sie in O-Notation an, wie oft die Vergleiche `if v1 == v2` durchgeführt werden. Begründen Sie Ihr Ergebnis.
- Machen Sie einen Vorschlag, wie diese Funktion optimiert werden kann. Erläutern Sie, inwiefern dieser eine bessere Komplexität hat.

Hinweis: Sie müssen keinen konkreten, vollständigen Algorithmus angeben.